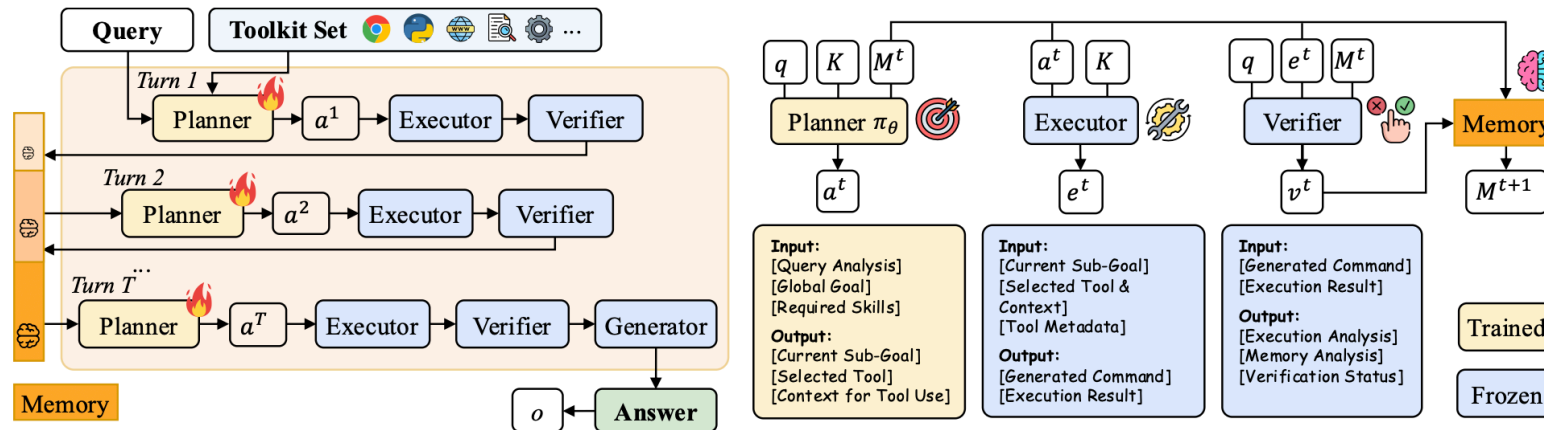


IN-THE-FLOW AGENTIC SYSTEM OPTIMIZATION FOR EFFECTIVE PLANNING AND TOOL USE

Zhuofeng Li^{*1,2}, Haoxiang Zhang^{*1,3}, Seungju Han¹, Sheng Liu¹, Jianwen Xie⁴,
Yu Zhang², Yejin Choi¹, James Zou^{†1}, Pan Lu^{†1}

¹Stanford University, ²Texas A&M University, ³UC San Diego, ⁴Lambda



(a) AgentFlow: In-the-Flow Agentic System

(b) In-the-Flow Rollout at Turn t

<https://openreview.net/pdf?id=Mf5AleTUVK>

Core idea

Train the planner of a modular agentic system directly inside the real multi-turn tool-use loop.

Why this paper matters

Modern RL reasoning methods work well when the task has a verifiable answer.

But real tool-using agents are harder:

- Many turns
- External tools
- Evolving memory
- Sparse final rewards
- Difficult credit assignment

AgentFlow moves from **model-level training** to **system-level training**.

Problem: monolithic tool-use LLMs

Many LLM agents use one model to do everything:

think → call tool → read result → think again → answer

This can become fragile when the horizon is long or the tool feedback changes the environment.

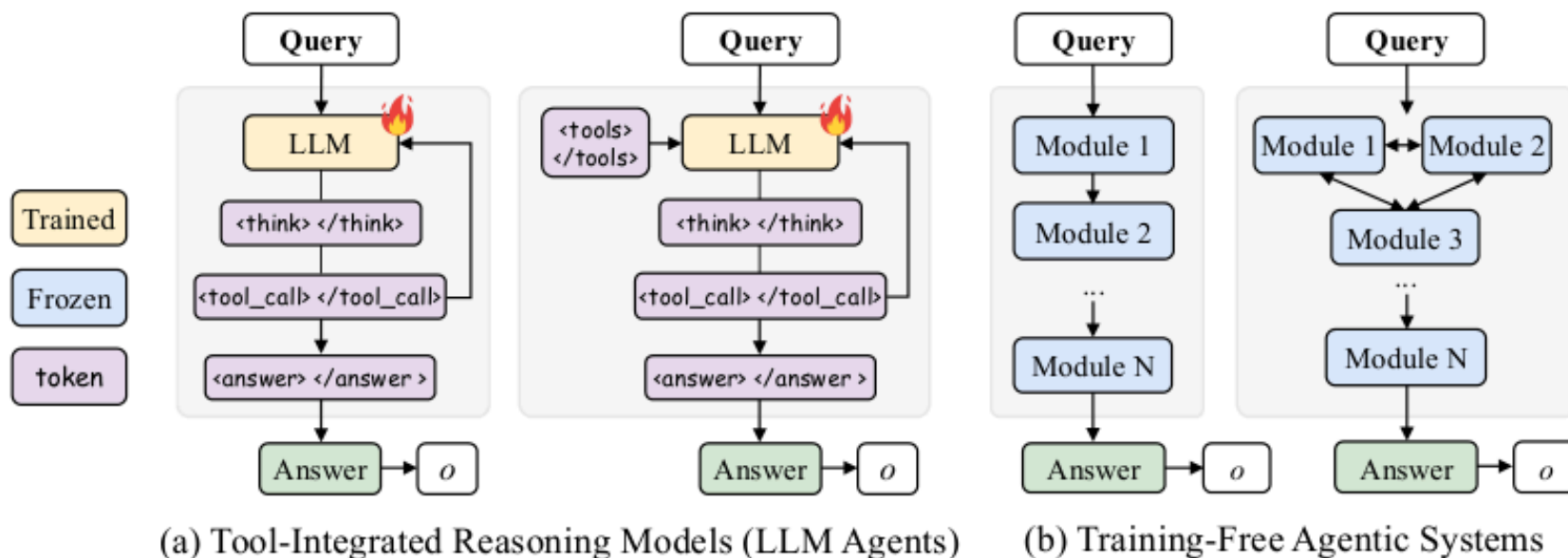


Figure 3: **Comparison of two paradigms of LLMs with tool use.** (a) Monolithic tool-integrated

AgentFlow architecture

AgentFlow decomposes the system into four modules:

Module	Responsibility
Planner	decides sub-goal, tool, and context
Executor	runs the selected tool
Verifier	checks whether the result is useful / sufficient
Generator	produces the final answer

Only the **planner** is optimized.

Multi-turn execution loop

At each turn, the planner observes:

state = query + toolset + memory

Then the system runs:

Planner → Executor → Verifier → Memory update

The loop stops when the answer is ready or the turn budget is exhausted.

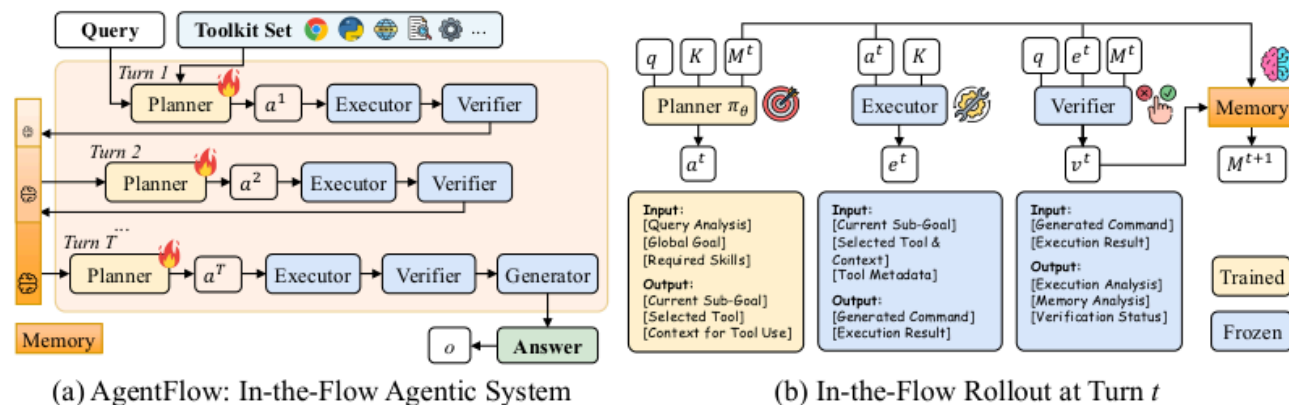


Figure 2: **(a)** Overview of AGENTFLOW, a trainable agentic system for in-the-flow planning and tool use. Four modules (planner, executor, verifier, generator) coordinate via a shared evolving memory M and toolset K , given a query q . The planner policy is optimized on-policy *inside* the system's multi-turn loop to enable adaptive, long-horizon reasoning. **(b)** A single state transition, showing the action a^t , execution result e^t , and verifier signal v^t that update the memory from M^t to M^{t+1} .

What “in-the-flow” means

The planner is not trained offline from fixed trajectories.

It is trained inside the actual agent loop:

```
plan → execute tool → verify → update memory → plan again
```

This exposes the planner to the real states it will face at inference time.

Flow-GRPO: training the planner

Flow-GRPO extends GRPO to multi-turn agentic trajectories.

For each query, the system samples multiple complete rollouts, computes final reward, and compares trajectories within the group.

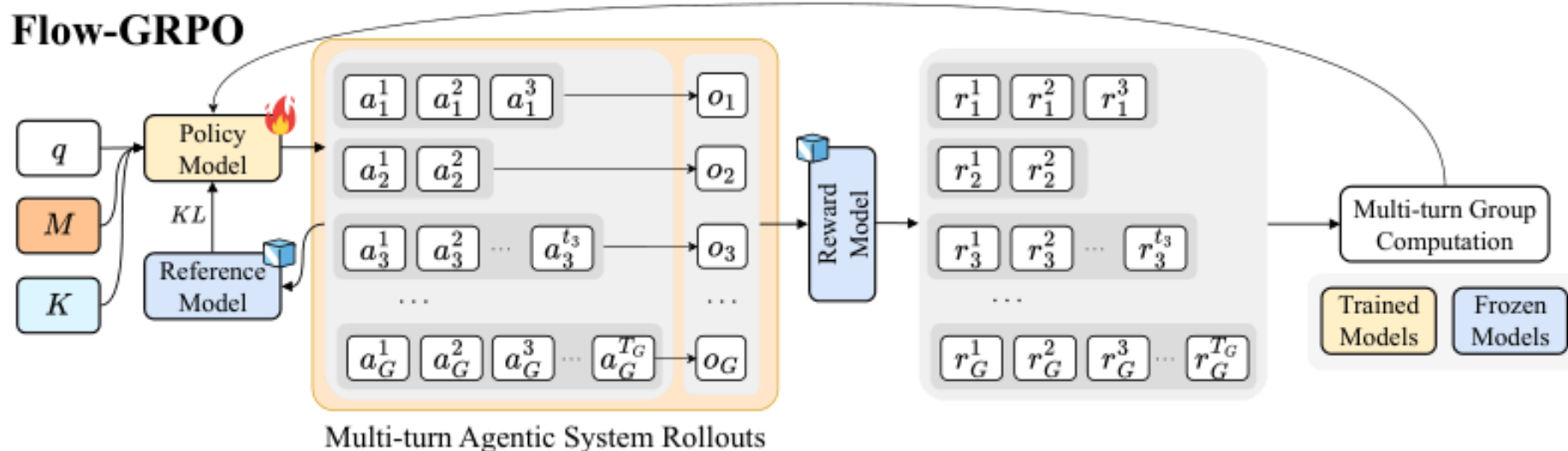


Figure 4: **Optimization for our proposed agentic system AGENTFLOW.** Given a query q , an evolving memory M , and a toolset K , the policy model generates actions that target sub-goals and

Sparse reward and credit assignment

The agent receives reward only after the final answer:

```
Action 1  
Action 2  
Action 3  
...  
Final answer → reward
```

The challenge: which action caused success or failure?

Flow-GRPO uses a simple strategy:

Broadcast the final trajectory-level reward to every planner action.

Group-normalized advantage

For each query, sample a group of trajectories.

$$\text{Advantage}_i = (\text{reward}_i - \text{mean}(\text{group rewards})) / \text{std}(\text{group rewards})$$

Intuition:

- Better-than-average trajectories get positive advantage
- Worse-than-average trajectories get negative advantage
- No separate value model is needed

Main results: search and agentic tasks

AgentFlow + Flow-GRPO improves over frozen AgentFlow and agentic baselines on search-intensive QA and GAIA.

Model	Size	Search Intensive						Agentic	
		Bamboogle	2Wiki	HotpotQA	Musique	Avg.	Δ	GAIA	Δ
Qwen-2.5-7B-Instruct	7B-Inst	12.0	23.0	21.0	6.0	15.5	$\uparrow 41.8$	3.2	$\uparrow 29.9$
Qwen-2.5-14B-Instruct	14B-Inst	21.6	26.7	20.0	8.0	19.1	$\uparrow 38.2$	5.5	$\uparrow 27.6$
Qwen-2.5-32B-Instruct	32B-Inst	24.0	26.7	27.0	6.0	20.9	$\uparrow 36.4$	9.5	$\uparrow 23.6$
Llama-3.3-70B-Instruct	70B-Inst	18.4	22.7	52.0	16.0	27.3	$\uparrow 30.0$	3.2	$\uparrow 29.9$
GPT-4o-mini (Hurst et al., 2024)	~8B	40.8	35.6	41.0	15.0	33.1	$\uparrow 24.2$	7.1	$\uparrow 26.0$
GPT-4o (Hurst et al., 2024)	~200B	68.8	49.5	54.0	24.0	49.1	$\uparrow 8.2$	17.3	$\uparrow 15.8$
Supervised Fine-Tuning (SFT)	7B-Inst	12.0	25.9	22.0	6.6	16.6	$\uparrow 40.7$	3.2	$\uparrow 29.9$
Iter-RetGen (Shao et al., 2023)	7B-Inst	36.8	33.6	37.4	17.8	31.4	$\uparrow 25.9$	3.9	$\uparrow 29.2$
Search-R1 (Jin et al., 2025)	7B-Inst	43.2	38.2	37.0	14.6	33.3	$\uparrow 24.0$	19.1	$\uparrow 14.0$
ZeroSearch (Sun et al., 2025)	7B-Base	27.8	35.2	34.6	18.0	28.9	$\uparrow 28.4$	16.5	$\uparrow 16.6$
ReSearch (Chen et al., 2025)	7B-Base	42.4	47.6	43.5	22.3	39.0	$\uparrow 18.3$	17.3	$\uparrow 15.8$
StepSearch (Wang et al., 2025d)	7B-Base	40.0	36.6	38.6	22.6	34.5	$\uparrow 22.8$	–	–
VerlTool (Jiang et al., 2025)	7B-Base	46.4	45.3	44.8	19.3	39.0	$\uparrow 18.3$	11.2	$\uparrow 21.9$
AutoGen (Wu et al., 2024)	7B-Inst	59.6	44.0	50.0	15.9	42.4	$\uparrow 14.9$	6.3	$\uparrow 26.8$
AGENTFLOW	7B-Inst	58.4	60.0	51.3	19.2	47.2	$\uparrow 12.1$	17.2	$\uparrow 15.9$
AGENTFLOW (w/ Flow-GRPO)	7B-Inst	69.6	77.2	57.0	25.3	57.3	–	33.1	–

Key reported gains:

- Search-intensive QA: **+14.9%**
- Agentic reasoning: **+14.0%**

Main results: math and science

Flow-GRPO also improves mathematical and scientific reasoning.

Model	Size	Math Reasoning					Scientific Reasoning			
		AIME24	AMC23	GameOf24	Avg.	Δ	GPQA	MedQA	Avg.	Δ
Qwen-2.5-7B-Instruct	7B-Inst	6.7	47.5	33.0	29.1	$\uparrow 22.5$	34.0	66.0	50.0	$\uparrow 13.5$
Qwen-2.5-14B-Instruct	14B-Inst	6.7	60.0	25.0	30.6	$\uparrow 21.0$	31.0	75.0	53.0	$\uparrow 10.5$
Llama-3.3-70B-Instruct	70B-Inst	6.7	47.5	31.0	28.4	$\uparrow 23.1$	35.0	67.0	51.0	$\uparrow 12.5$
Llama-3.1-405B-Instruct	405B-Inst	26.7	47.5	23.0	32.4	$\uparrow 19.1$	30.0	62.0	46.0	$\uparrow 17.5$
GPT-4o-mini (Hurst et al., 2024)	$\sim 8B$	13.3	57.5	16.0	28.9	$\uparrow 22.6$	27.0	66.0	46.5	$\uparrow 17.0$
GPT-4o (Hurst et al., 2024)	$\sim 200B$	13.3	60.0	32.0	35.1	$\uparrow 16.4$	31.0	60.0	45.5	$\uparrow 18.0$
Supervised Fine-Tuning (SFT)	7B-Inst	6.7	47.5	33.0	29.1	$\uparrow 22.5$	34.0	66.0	50.0	$\uparrow 13.5$
SimpleRL-reason (Zeng et al., 2025b)	7B-Base	16.7	60.0	33.0	36.6	$\uparrow 15.0$	45.0	65.0	50.0	$\uparrow 13.5$
Open-Reasoner-Zero (Hu et al., 2025a)	7B-Base	16.7	54.9	32.0	34.5	$\uparrow 17.0$	34.0	54.0	44.0	$\uparrow 19.5$
General-Reasoner (Ma et al., 2025)	7B-Base	13.3	55.0	33.0	33.8	$\uparrow 17.7$	35.5	61.0	48.3	$\uparrow 15.2$
Luffy (Yan et al., 2025)	7B-Inst	30.7	44.8	33.0	36.2	$\uparrow 15.3$	34.0	77.0	55.5	$\uparrow 8.0$
TIR (Yang et al., 2024b)	7B-Inst	10.0	50.0	33.0	31.0	$\uparrow 20.5$	42.0	76.8	59.4	$\uparrow 4.1$
ToRL (Li et al., 2025b)	7B-Inst	20.0	60.0	31.0	37.0	$\uparrow 14.5$	35.0	76.5	55.8	$\uparrow 7.7$
AutoGen (Wu et al., 2024)	7B-Inst	13.3	57.5	24.0	31.6	$\uparrow 19.9$	42.0	72.0	57.0	$\uparrow 6.5$
AGENTFLOW	7B-Inst	16.7	47.4	31.0	31.7	$\uparrow 19.8$	37.0	76.0	56.5	$\uparrow 7.0$
AGENTFLOW (w/ Flow-GRPO)	7B-Inst	40.0	61.5	53.0	51.5	–	47.0	80.0	63.5	–

Table 2: Accuracy comparison of mathematical and scientific reasoning tasks. As the same in Table 1, AutoGen and AGENTFLOW use Qwen-2.5-7B-Instruct for the LLM-powered tools.

Key reported gains:

- Math reasoning: **+14.5%**
- Scientific reasoning: **+4.1%**

Why stronger frozen planners are not enough

The paper compares several planner strategies:

- Frozen Qwen-2.5-7B planner
- Frozen GPT-4o planner
- Offline SFT planner
- Flow-GRPO trained planner

Planner Model	Training	Bamboogle	2Wiki	GAIA	AIME24	AMC23	GameOf24	Avg.
Qwen-2.5-7B	Frozen	58.4	60.0	17.2	16.7	47.4	31.0	38.5
GPT-4o	Frozen	65.0 ↑ 6.6	70.0 ↑ 10.0	23.6 ↑ 6.4	16.7 ↑ 0.0	48.7 ↑ 1.3	42.0 ↑ 11.0	44.3 ↑ 5.8
Qwen-2.5-7B	SFT	30.4 ↓ 28.0	32.7 ↓ 27.3	6.3 ↓ 10.9	3.3 ↓ 13.4	37.5 ↓ 9.9	7.0 ↓ 24.0	19.5 ↓ 19.0
Qwen-2.5-7B	Flow-GRPO	69.6 ↑ 11.2	77.2 ↑ 17.2	33.1 ↑ 15.9	40.0 ↑ 23.3	61.5 ↑ 14.1	53.0 ↑ 22.0	55.7 ↑ 17.2

Table 3: Performance comparison of AGENTFLOW across different training methods.

Main lesson:

| *A stronger planner helps, but training in the real system dynamics matters more.*

Case study: self-correction

Before Flow-GRPO, the agent repeats similar failing actions.

After Flow-GRPO, it explores a new solution path and recovers.

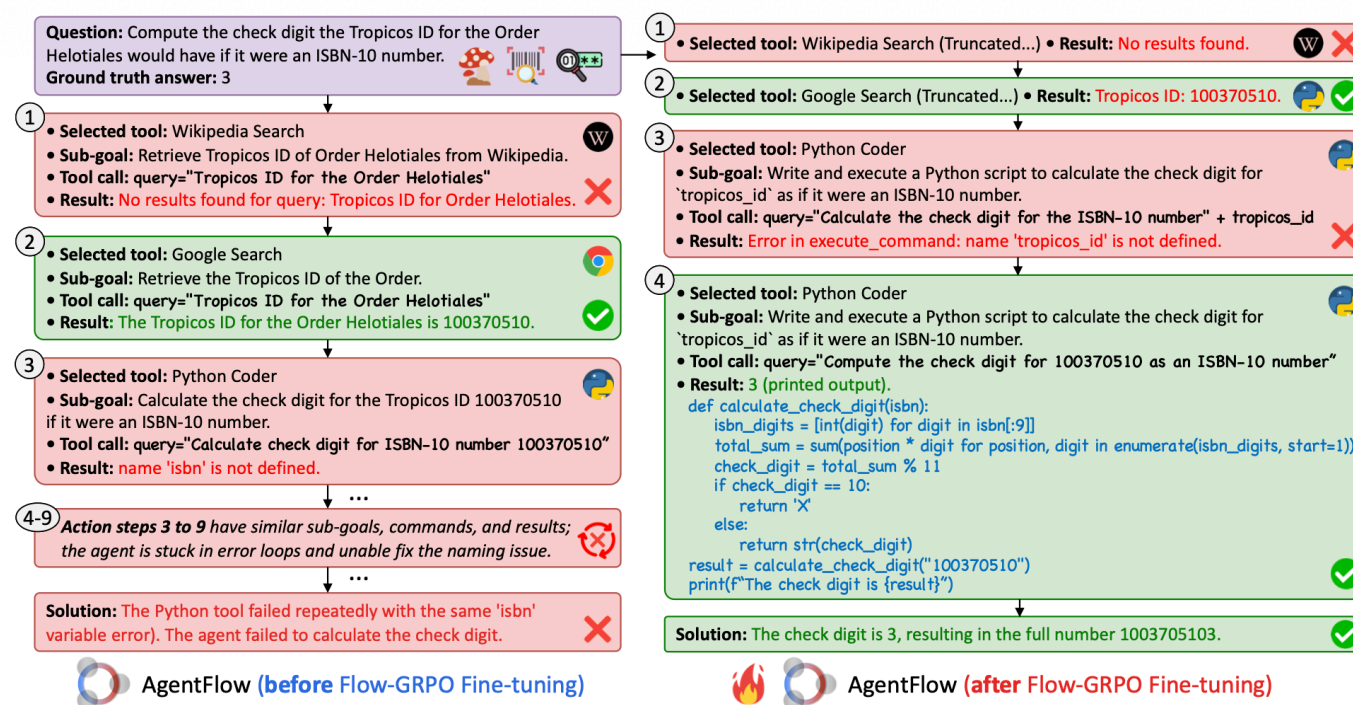


Figure 7: **One case study example.** Initially failed with repetitive errors (left), AGENTFLOW, trained with Flow-GRPO, explores a new solution pathway at turn 4 after two failed attempts (right).

This suggests the planner learns better tool-use adaptation and error recovery.

Training and scaling behavior

Flow-GRPO improves reward over training and scales with model size and turn budget.

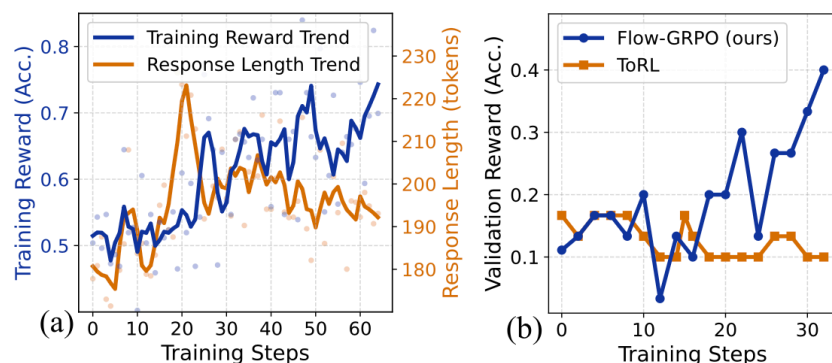


Figure 8: Training dynamics and efficiency of Flow-GRPO.

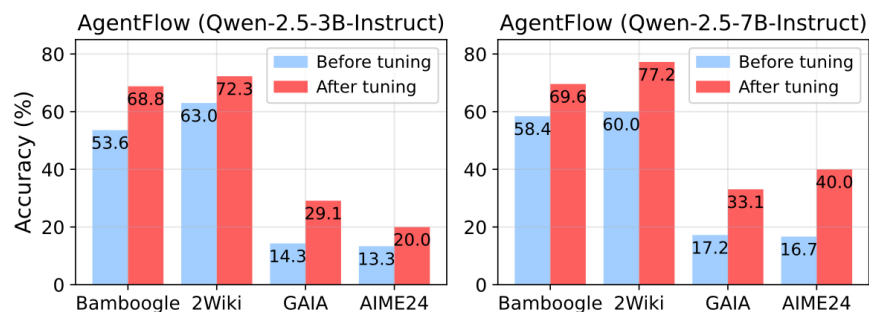


Figure 9: Flow-GRPO fine-tuning offers consistent gains on AGENTFLOW as the backbone model size scales from 3B to 7B.

Turns ($T_{\max}$)	3	5	7	10
2Wiki	2.22	3.18	3.81	4.44
GameOf24	1.63	2.12	2.36	2.67
AIME24	1.63	1.63	1.86	1.90
GAIA	2.43	3.46	4.28	5.42

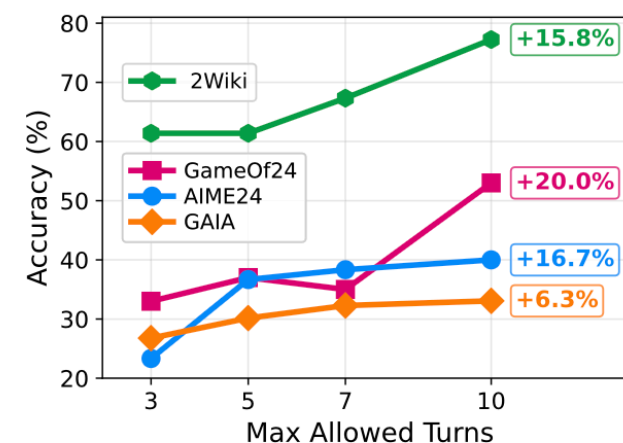


Figure 10: Average turns and accuracy with increased $T_{\max}$.

Practical takeaways

For real-world agentic systems:

- Log full trajectories, not just final answers
- Separate planner from executor / verifier / generator
- Optimize for task-level outcome reward
- Train inside the real tool-use loop
- Track tool calls, turn count, retries, and self-correction behavior

Final takeaway

AgentFlow reframes the problem:

From: training a reasoning model
To: training an agentic system

The most important insight:

| *Agent training should happen at the system level, inside the workflow where the agent actually operates.*